

OOP in Java

BCA — Semester 3 — 2024 — Answer Sheet

Subject Code: CACS204

Group B

Attempt any SIX questions. [6×5=30]

2. What is constructor? Explain the role of constructor in Java class by considering a suitable example and also list the type of constructor.

[1+3+1]

Answer: A **constructor** is a special method in Java that is automatically invoked when an object of a class is created. It has the same name as the class and does not have a return type—not even void. **Role of Constructor in Java:**

1. **Object Initialization:** Constructors initialize the instance variables of a class at the time of object creation.
2. **Memory Allocation:** They help in allocating memory to the object on the heap.
3. **Default Values:** If no constructor is defined, Java provides a default constructor that initializes variables with default values (0, null, false).
4. **Code Reusability:** Constructors can call other constructors using `this()`, promoting code reuse. **Example:**

```
class Student { int id; String name; // Parameterized Constructor Student(int i, String n) { id = i; name = n; } void display() { System.out.println("ID: " + id + ", Name: " + name); } public static void main(String[] args) { Student s1 = new Student(101, "Ram"); s1.display(); } }
```

Types of Constructor:

1. **Default Constructor:** No parameters; provided by Java if none is written.
2. **Parameterized Constructor:** Accepts one or more parameters to initialize object with specific values.
3. **Copy Constructor:** Copies values from one object to another (not built-in in Java but can be created manually).

3. Define inheritance. Explain types of inheritance with suitable example.

[1+4]

Answer: **Inheritance** is an Object-Oriented Programming concept where one class (child/subclass) acquires the properties and behaviors (fields and methods) of another class (parent/superclass). It promotes code reusability and establishes an "is-a" relationship. **Types of Inheritance in Java:** 1. **Single Inheritance:** A class inherits from one parent class. `class Animal { void eat() { System.out.println("Eating..."); } } class Dog extends Animal { void bark() { System.out.println("Barking..."); } }` 2. **Multilevel Inheritance:** A class inherits from a class which itself inherits from another class. `class Animal { void eat() {} } class Dog extends Animal { void bark() {} } class Puppy extends Dog { void weep() {} }` 3. **Hierarchical Inheritance:** Multiple classes inherit from a single parent class. `class Animal { void eat() {} } class Dog extends Animal { void bark() {} } class Cat extends Animal { void meow() {} }` 4. **Multiple Inheritance (via Interfaces):** A class implements multiple interfaces. `interface Flyable { void fly(); } interface Swimmable { void swim(); } class Duck implements Flyable, Swimmable { public void fly() { System.out.println("Flying"); } public void swim() { System.out.println("Swimming"); } }` 5. **Hybrid Inheritance:** A combination of two or more types of inheritance (achieved using interfaces in Java).

4. Define Package. Explain steps in creation and implementation of package with example.

[1+4]

Answer: A **package** in Java is a namespace that organizes a set of related classes and interfaces. It helps avoid naming conflicts, provides access protection, and makes searching and locating classes easier. **Steps to Create and Implement a Package:**

Step 1 — Declare the Package: Use the package keyword at the top of the Java source file. `package mypackage;` **Step 2 — Create Directory Structure:** The directory structure must match the package name. For mypackage, create a folder named mypackage. **Step 3 — Place Source File:** Save the Java file inside the package directory. `// mypackage/Student.java package mypackage; public class Student { public void display() { System.out.println("Hello from mypackage!"); } }` **Step 4 — Compile the Package:** `javac -d . mypackage/Student.java` The `-d .` option creates the package directory automatically. **Step 5 — Use the Package (Import and Implement):** `// TestPackage.java import mypackage.Student; public class TestPackage { public static void main(String[] args) { Student s = new Student(); s.display(); } }` **Compile and Run:** `javac TestPackage.java java TestPackage` **Types of Packages:**

- **Built-in packages:** `java.lang`, `java.util`, `java.io`, etc.
- **User-defined packages:** Created by the programmer as shown above.

5. What is multithreading? How does Java support inter-thread communication? Explain with example.

[1+4]

Answer: Multithreading is a process of executing multiple threads simultaneously within a single program. A **thread** is the smallest unit of processing that can be scheduled by the operating system. Multithreading improves CPU utilization, enables concurrent execution, and makes applications more responsive.

Inter-Thread Communication in Java:

Java supports inter-thread communication using three methods from the Object class:

- `wait()` — Causes the current thread to release the lock and wait until another thread invokes `notify()` or `notifyAll()`.
 - `notify()` — Wakes up a single thread that is waiting on the object's monitor.
 - `notifyAll()` — Wakes up all threads waiting on the object's monitor. These methods must be called from a synchronized context. **Example — Producer-Consumer Problem:**
- ```
class SharedResource { boolean flag = false; int data; synchronized void produce(int value) throws InterruptedException { while (flag) { wait(); } data = value; System.out.println("Produced: " + data); flag = true; notify(); } synchronized void consume() throws InterruptedException { while (!flag) { wait(); } System.out.println("Consumed: " + data); flag = false; notify(); } } class Producer extends Thread { SharedResource sr; Producer(SharedResource sr) { this.sr = sr; } public void run() { try { for (int i = 1; i <= 5; i++) sr.produce(i); } catch (InterruptedException e) { e.printStackTrace(); } } } class Consumer extends Thread { SharedResource sr; Consumer(SharedResource sr) { this.sr = sr; } public void run() { try { for (int i = 1; i <= 5; i++) sr.consume(); } catch (InterruptedException e) { e.printStackTrace(); } } } public class InterThreadDemo { public static void main(String[] args) { SharedResource sr = new SharedResource(); new Producer(sr).start(); new Consumer(sr).start(); } }
```

## 6. How to handle multiple catch blocks for a nested try block? Explain with an example.

[5]

**Answer:** In Java, a **nested try block** is a try block inside another try block. This is useful when different sections of code may throw different exceptions. Each inner or outer try block can have its own set of **multiple catch blocks** to handle specific exception types. The JVM checks catch blocks from top to bottom and executes the first matching one. **Example — Nested Try with Multiple Catch Blocks:**

```
public class NestedTryCatch { public static void main(String[] args) { try { // Outer try block int[] arr = {10, 20, 30}; System.out.println("Outer try begins"); try { // Inner try block int result = arr[2] / 0; // ArithmeticException System.out.println(result); } catch (ArithmeticException e) { System.out.println("Inner catch: " + e.getMessage()); } try { // Another inner try block System.out.println(arr[5]); // ArrayIndexOutOfBoundsException } catch (ArrayIndexOutOfBoundsException e) { System.out.println("Inner catch 2: " + e.getMessage()); } String s = null; System.out.println(s.length()); // NullPointerException } catch (NullPointerException e) { System.out.println("Outer catch: " + e.getMessage()); } catch (Exception e) { System.out.println("General catch: " + e.getMessage()); } finally { System.out.println("Finally block executed"); } } }
```

**Output:** Outer try begins Inner catch: / by zero Inner catch 2: Index 5 out of bounds for length 3 Outer catch: null Finally block executed **Key Points:**

- The inner catch handles exceptions raised in its corresponding inner try.
- If the inner catch cannot handle the exception, it propagates to the outer catch.
- Multiple catch blocks should be ordered from most specific to most general.
- The finally block always executes regardless of exceptions.

**7. What is difference between String and StringBuffer and explain 3 methods of String and StringBuffer class with example.**

[2+3]

**Answer: Difference between String and StringBuffer:** Aspect String StringBuffer Mutability Immutable (cannot be changed once created) Mutable (can be modified) Memory Creates a new object for every modification Modifies the same object, saving memory Thread Safety Thread-safe due to immutability Thread-safe (all methods are synchronized) Performance Slower for frequent modifications Faster for frequent modifications Storage Stored in String Constant Pool Stored in Heap memory

**3 Methods of String Class with Example:** String s = "Hello World"; // 1. length() — Returns the number of characters int len = s.length(); // Output: 11 // 2. substring(int beginIndex, int endIndex) — Extracts a portion String sub = s.substring(0, 5); // Output: "Hello" // 3. equalsIgnoreCase(String another) — Compares ignoring case boolean result = s.equalsIgnoreCase("hello world"); // Output: true

**3 Methods of StringBuffer Class with Example:** StringBuffer sb = new StringBuffer("Hello"); // 1. append(String str) — Adds text at the end sb.append("World"); // sb becomes "Hello World" // 2. insert(int offset, String str) — Inserts text at specified position sb.insert(5, ","); // sb becomes "Hello, World" // 3. reverse() — Reverses the character sequence sb.reverse(); // sb becomes "dlroW ,olleH" System.out.println(sb);

**8. Write short notes on: (any two) i. JDBC Driver ii. Access Modifiers iii. JDK, JRE and JVM**

[2.5+2.5]

**Answer: i. JDBC Driver:**

JDBC (Java Database Connectivity) Driver is a software component that enables Java applications to interact with databases. It translates Java method calls into database-specific commands. **Types of JDBC Drivers:**

1. **Type 1 — JDBC-ODBC Bridge:** Uses ODBC driver to connect to the database. Platform dependent and slow.
2. **Type 2 — Native-API Driver:** Uses database-specific client-side libraries. Faster than Type 1 but requires native code.
3. **Type 3 — Network Protocol Driver:** Uses a middleware server to translate JDBC calls into database protocol. Platform independent.
4. **Type 4 — Thin Driver (Pure Java):** Directly converts JDBC calls into database protocol. Platform independent and fastest. Most commonly used today.

**ii. Access Modifiers:**

Access modifiers in Java control the visibility and accessibility of classes, methods, and variables.

**Modifier Class Package Subclass World** public Yes Yes Yes Yes protected Yes Yes Yes No default (no modifier) Yes Yes No No private Yes No No No • **public:** Accessible everywhere.

- **protected:** Accessible within the package and by subclasses.
- **default:** Accessible only within the same package.
- **private:** Accessible only within the declaring class.

**iii. JDK, JRE and JVM:**

**JVM (Java Virtual Machine):** An abstract machine that provides the runtime environment for Java bytecode. It is platform-dependent and responsible for loading, verifying, and executing bytecode. **JRE (Java Runtime Environment):** A software package that provides the libraries, JVM, and other components required to run Java applications. It does not include development tools. End users only need JRE to run Java programs. **JDK (Java Development Kit):** A complete software development kit for Java. It includes JRE + development tools such as compiler (javac), debugger (jdb), archiver (jar), and documentation tools (javadoc). Developers need JDK to write and compile Java programs. **Relationship:** JDK  $\supset$  JRE  $\supset$  JVM

---

## Group C

Attempt any TWO questions. [2×10=20]

9. Define array and multi dimensional array. Write a program to read two m X n matrices, perform multiplication operation and store result in third matrix.

[3+7]

**Answer: Array:** An array is a collection of elements of the same data type stored in contiguous memory locations. It is accessed using an index and provides a way to store multiple values under a single variable name. **Multi-Dimensional Array:** A multi-dimensional array is an array of arrays. The most common form is the two-dimensional array, which represents data in rows and columns (like a matrix). In Java, a 2D array is declared as `int[][] arr = new int[rows][cols];`. **Java Program for Matrix Multiplication:**

```
import java.util.Scanner;
public class MatrixMultiplication {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 System.out.print("Enter number of rows (m): ");
 int m = sc.nextInt();
 System.out.print("Enter number of columns (n): ");
 int n = sc.nextInt();
 int[][] matrix1 = new int[m][n];
 int[][] matrix2 = new int[m][n];
 int[][] result = new int[m][n];
 System.out.println("Enter elements of first matrix:");
 for (int i = 0; i < m; i++)
 for (int j = 0; j < n; j++)
 matrix1[i][j] = sc.nextInt();
 System.out.println("Enter elements of second matrix:");
 for (int i = 0; i < m; i++)
 for (int j = 0; j < n; j++)
 matrix2[i][j] = sc.nextInt();
 // Matrix multiplication
 for (int i = 0; i < m; i++) {
 for (int j = 0; j < n; j++) {
 result[i][j] = 0;
 for (int k = 0; k < n; k++) {
 result[i][j] += matrix1[i][k] * matrix2[k][j];
 }
 }
 }
 System.out.println("Resultant Matrix:");
 for (int i = 0; i < m; i++) {
 for (int j = 0; j < n; j++) {
 System.out.print(result[i][j] + " ");
 }
 System.out.println();
 }
 sc.close();
 }
}
```

**Note:** For proper matrix multiplication, the number of columns in the first matrix should equal the number of rows in the second matrix. The program above assumes square matrices of size m×n for simplicity. For general multiplication (m×n and n×p), adjust dimensions accordingly.

---

10. Write a program to create the following simple GUI based application. If user press the Submit button, your program should store the information in a file named "exam.txt" only when he accepts the terms and condition otherwise it should display a message "please accept the terms and condition first".

[6+3+1]

**Answer: Java GUI Program using Swing:**

```
import javax.swing.*; import java.awt.*; import java.awt.event.*;
import java.io.*; public class ExamForm extends JFrame implements ActionListener { JTextField nameField,
emailField; JCheckBox termsCheck; JButton submitButton; JLabel messageLabel; public ExamForm() {
setTitle("Exam Registration Form"); setSize(400, 300); setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
setLayout(new GridLayout(6, 2, 10, 10)); // Name add(new JLabel("Name:")); nameField = new JTextField();
add(nameField); // Email add(new JLabel("Email:")); emailField = new JTextField(); add(emailField); // Terms
checkbox termsCheck = new JCheckBox("I accept the terms and conditions"); add(termsCheck); add(new
JLabel("")); // Submit button submitButton = new JButton("Submit"); submitButton.addActionListener(this);
add(submitButton); // Message label messageLabel = new JLabel("");
messageLabel.setForeground(Color.RED); add(messageLabel); setVisible(true); } public void
actionPerformed(ActionEvent e) { if (e.getSource() == submitButton) { if (!termsCheck.isSelected()) {
messageLabel.setText("please accept the terms and condition first"); } else { String name =
nameField.getText(); String email = emailField.getText(); try { FileWriter fw = new FileWriter("exam.txt", true);
BufferedWriter bw = new BufferedWriter(fw); bw.write("Name: " + name + ", Email: " + email); bw.newLine();
bw.close(); messageLabel.setForeground(Color.GREEN); messageLabel.setText("Information saved
successfully!"); } catch (IOException ex) { messageLabel.setText("Error writing to file: " + ex.getMessage()); } }
} } public static void main(String[] args) { new ExamForm(); } }
```

**Explanation:**

- The form uses JTextField for name and email input.
- JCheckBox allows the user to accept terms and conditions.
- When Submit is clicked, actionPerformed() checks if the checkbox is selected.
- If not selected, it displays "please accept the terms and condition first".
- If selected, it writes the information to exam.txt using FileWriter and BufferedWriter.
- The file is opened in append mode (true) so new entries are added without overwriting existing data.

---

*Note: These answers are prepared for educational purposes. Students are encouraged to verify with official textbooks and course materials.*